

Reviewer Pack — Minimal Quaternionic Norm Correlation with Circuit Monotone ...

Entry 0fff51ec5192 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 20:09:12 UTC

Minimal Quaternionic Norm Correlation with Circuit Monotone Width

Entry ID: 0fff51ec5192 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 20:09:12 UTC

1. Conjecture

Field A (mathematical branch): Quaternionic Analysis **Field B** (complexity object): Boolean Circuits

Statement:

The minimal quaternionic norm of the Hamiltonian matrix associated with a CNF formula is linearly correlated with its circuit monotone width, such that $\|H(\varphi)\| = \Theta(w(\varphi))$, where $H(\varphi)$ is the Hamiltonian matrix derived from φ and $w(\varphi)$ is the circuit monotone width of φ .

Rationale (proposer's reasoning):

Quaternionic analysis provides a framework for extending complex numbers to four-dimensional space. The minimal quaternionic norm could potentially capture geometric properties that are not evident in standard algebraic structures, which might reveal new insights into circuit complexity.

Taxonomy category: Quaternionic_Analysis × Boolean_Circuits (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 20b8270af84cb5f1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\|H(\varphi)\|$ and $w(C(\varphi))$ exceeds 0.7 for at least 24 of the 30 seeds, with no seed producing a correlation coefficient less than 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quaternionic norm" AND "circuit monotone width" - "Hamiltonian matrix" [JOUR] AND CNF formula [JOUR] - "minimal quaternionic norm" [SUBS] AND circuit complexity [JOUR]

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        clauses = []
        for _ in range(m):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(c == 0 for c in clause):
                clause[random.randint(0, n - 1)] = random.choice([-1, 1])
            clauses.append(clause)
        return clauses

    def hamiltonian_matrix(cnf):
        n = len(cnf[0])
        H = [[Fraction(0) for _ in range(n)] for _ in range(n)]
        for clause in cnf:
            for i in range(n):
                if clause[i] != 0:
                    for j in range(i + 1, n):
                        if clause[j] != 0:
                            H[i][j] += Fraction(1)
                            H[j][i] += Fraction(1)
        return H

    def min_quaternionic_norm(H):
        n = len(H)
        I = [[Fraction(1) if i == j else Fraction(0) for j in range(n)] for i in range(n)]
        A = [H[i] + I[i] for i in range(n)]
        B = [I[i] - H[i] for i in range(n)]
```

```

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for k in range(i + 1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        for j in range(n):
            A[i][j] /= pivot
        for k in range(n):
            if k != i:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]
    return A

A = gaussian_elimination(A)
B = gaussian_elimination(B)

norm_A = sum(sum(abs(a) for a in row) for row in A)
norm_B = sum(sum(abs(b) for b in row) for row in B)
return min(norm_A, norm_B)

def monotone_width(cnf):
    n = len(cnf[0])
    width = 0
    for clause in cnf:
        width = max(width, sum(1 for literal in clause if literal != 0))
    return width

results = []
for _ in range(30):
    n = random.randint(5, 40)
    m = random.randint(n, n * 2)
    cnf = generate_cnf(n, m)
    H = hamiltonian_matrix(cnf)
    norm_H = min_quaternionic_norm(H)
    w_C = monotone_width(cnf)
    results.append((norm_H, w_C))

if not results:
    return {
        "metric_name": "min_quaternionic_norm",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

norm_values = [norm for norm, _ in results]
width_values = [width for _, width in results]

```

```

mean_norm = sum(norm_values) / len(norm_values)
std_norm = math.sqrt(sum((x - mean_norm) ** 2 for x in norm_values) / len(norm_values))
correlation = sum(norm * width for norm, width in results) / (len(results) * mean_norm * mean_norm)

return {
    "metric_name": "min_quaternionic_norm",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(n for n, _ in results),
    "conjecture_holds": correlation >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_corr = sum(result["metric_value"] for result in results) / len(results)
    std_corr = math.sqrt(sum((result["metric_value"] - mean_corr) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_corr} std={std_corr} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_corr} std={std_corr} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_too_low\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8da82118.py", line 122, in <module>
    trial_result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8da82118.py", line 86, in run_trial
    norm_H = min_quaternionic_norm(H)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8da82118.py", line 46, in min_quaternionic_norm
    B = [I[i] - H[i] for i in range(n)]
    ~~~~~^~~~~~

```

TypeError: unsupported operand type(s) for -: 'list' and 'list'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the Pearson correlation coefficient for the conjecture. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14285
2	propose	ollama_remote	glm4:latest	0	0	12931
3	preregistration	ollama_remote	glm4:latest	0	0	9113
4	novelty	ollama_remote	glm4:latest	0	0	8368
5	novelty	ollama_remote	glm4:latest	0	0	10628
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18084
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25312
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19877
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18043
10	judge	ollama_remote	glm4:latest	0	0	12181

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148822 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/0fff51ec5192.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/0fff51ec5192.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0fff51ec5192.tar.gz (if generated)